Spire — Test Plan

(1) the existing test sets and how to run them; (2) how to test manually — where to focus, how to run, how to build new prompts, how to record results.

> Run commands are as-documented — confirm script names/flags against the repo.

Part 1 — Existing test sets

12 prompt sets, 252 prompts, run nightly — the regression baseline (known prompts, known-good shapes).

Set	#	Covers
`00_dipstick`	6	Smoke test — one of each shape. Run first to confirm the system is up.
`01_supplier`	40	Supplier lists, classifications, country/BU filters, dates, columns. P2P core.
`02_purchase_order`	9	PO header + children (lines, schedules, distributions). QA-01.
`03_ar`	16	AR invoices, lines, tax lines, credit memos, receipts. QA-01 + QA-03.
`04_columns`	8	Column projection (“only name and status”). COL-01.
`05_refusals`	9	Prompts that **should** refuse — all 9 do, by design.
`06_slow`	15	Edge-traversal filters (invoice-amount, open-POs). Slowest (~68s/prompt). QA-02.
`07_alacarte`	12	Mixed column requests + filters combined.
`08_lifecycle`	31	PO lifecycle — received/invoiced/paid, “is it complete”. QA-07.
`10_refusal_examples`	19	Prompts that **should answer** — all 19 do (counterpart to 05).
`11_column_vocabulary`	53	Column synonyms — `vendor name`, `who columns`, `created by`. COL-02/03.
`qa_testcases`	34	Edge cases — under-specified, contextual, unsupported. Mostly refuses (22/34).

How to run

```
# One set (sets live in scripts/prompts/ as <set>.txt)
python scripts/run_prompt_set.py --prompt-file scripts/prompts/01_supplier.txt 2>&1 | tee t.txt

# One prompt, with routing visible
python scripts/run_graph.py --prompt "How much of PO 2144 was received?" --show-planner

# All sets (nightly batch)
python scripts/run_overnight.py --outdir overnight_20260808_postfix
```

Diagnostics are already on — `show_all = true` in `pyproject.toml`, so every run prints planner/synthesizer/records/tokens/timings. No `--show-*` needed. Override per run with `--no-show-planner`, per machine with `SPIRE_SHOW_*=1`.

Save every run: pipe through `tee` — `2>&1 | tee t.txt` (overwrite) or `tee -a` (append). `2>&1` keeps crash messages in the file. Never a bare `> t.txt`. The batch runner already saves per-set files under `--outdir`.

Exit 0 means “it ran”, NOT “the answer is correct.” The runner’s tally is not a verdict — every answer must be read against the pod. That’s Part 2.

Part 2 — Manual testing

Goal: break Spire. The sets prove the *known* prompts work; manual testing finds the *unknown* failures — the phrasings nobody scripted. You know the Fusion data — use it. **The worst bug is a wrong number that renders beautifully.** Catch Spire agreeing with itself but disagreeing with the pod.

How to run one prompt: run it (`run_graph.py --prompt "..." 2>&1 | tee t.txt`) → read the answer → verify against the pod (SQL/BI or Fusion UI) → log it.

What a correct answer looks like

- **Sample:** `Showing 25 of 429 suppliers.` — a 25-row sample of a larger set.
- **Honest denominator:** `Examined 25 of 432. Of those 25, 10 match.` — never “10 of 432”.
- **Tip block:** a `── Tip ──` narrowing hint — only when the answer is a sample.
- **Parent + child:** `PO 162352 — lines:` + an indented row table — never `Lines: 2`.
- **Refusal:** a `Reason / Solution / Example` block + a type. A refusal can be the *correct* answer — judge whether the reason is accurate.

Before you log a mismatch — triage it

You verify over SQL/UI; Spire reads over REST. They don’t always agree, and that’s not always a bug. Classify before logging:

- **REAL** — genuinely wrong (bad number, lost column, wrong refusal). **Log it.**
- **METHOD** — REST-vs-SQL, not a defect (e.g. Spire samples 25 → “of 432” while `COUNT(*)`=432; supplier count drifts 429–432). **Not a bug.**
- **KNOWN** — already deferred (see below). **Don’t re-report.**

If unsure, mark METHOD/unsure and move on — over-logging noise costs more than it finds.

Where to focus — by area

Own your domain. Run a few baseline prompts to learn the shape, then build your own and push the edges.

- **A · Suppliers** — lists, classifications, country/BU filters, columns. *Focus:* does the honest denominator hold (“N of 25” never becomes “N of 432”)? Try a filter with **zero** matches. *Known:* `supplier sites for X` returns addresses, not sites.
- **B · POs + Receipts** — headers, lines/schedules/distributions, lifecycle, receipts. *Focus:* child rows as a **table** not `Lines: 2`; received/invoiced/paid match the pod; receipts render (not “none”). *Known:* lifecycle needs the order number (162352), not the header id

(2144).

- **C · AR Invoices + Credit Memos** — invoices, lines, tax lines, memos. *Focus:* invoice+lines as child tables; multi-currency totals **refused honestly** (“spans 4 currencies”), not summed.
- **D · AR Accounts** — accounts, balances, exposure rankings. *Focus:* ranking totals match the pod; negative/zero balances shown honestly. *Known real (do not re-report):* -206,448.85 credit balance, 165M exposure.
- **Everyone · Refusals & Columns** — is the refusal **reason accurate** or a generic “narrow by country/status” template? Request a real Oracle field — does it resolve? Mix **one valid + one fake** — is the valid column kept?

Build your own prompts

Pick a behaviour → **predict** the answer before running → vary the phrasing (synonyms, word order, extra/missing filters — failures hide here) → run and compare to pod → push a boundary (zero matches, one vs many children, empty date range) → log divergences. For a reusable set, add prompts to `scripts/overnight_sets.yaml` under a new name.

Collect results

One row per prompt, one table per tester, shared back at end of session.

#	Prompt	Expected shape	Pod says	Result	Evidence
1				REAL / METHOD / KNOWN / PASS	number + source

A finding needs: exact prompt, what Spire returned, what the pod holds, how you checked, triage class. Screenshot of answer + query is ideal. Regroup at the end — walk REAL findings together, drop METHOD/KNOWN.